1. Every protocol needs a name - what do you want to call this chat protocol?
   Levorat at the Park (LaP)

2. What is your complete CHAT message here? What headers did you use (if any) and how were they used?

   Our chat message was formatted as follows:

   ```
   CHAT\r\n
   Type: MESSAGE\r\n\r\n
   {message}\r\n
   ```

   - CHAT indicates the type of message being sent (ex: bridge, acks, register, etc)
     - We implemented it to keep consistent with the other messages that were being sent around, such as `/register` or `/bridge`
   - The `Type` header differentiates between "regular" MESSAGE messages and QUIT messages
   - The {message} section contains the actual message that is being sent over to the other client

3. **At what layer of the TCP/IP protocol stack is the Chat Protocol? Explain your answer.**

   The Chat Protocol operates at the Application Layer of the TCP/IP stack because it manages application-specific tasks like message formatting (REGISTER, BRIDGE, CHAT) and peer communication.

   It uses the Tr r reliable data transfer. Its primary role is to provide user-facing communication features.

4. Referring to the textbook Section 2.1.1:
   a. Looking over the architecture in this project, would you characterize it as Client-Server or something else? Explain your answer.
      The primary architecture is a client-server architecture, but during the chat service, it can be characterized as a Peer-to-Peer architecture. As the clients register, bridge, and request for id, a centralized server handles those requests and goes on to facilitate peer communication. However, once the clients begin their Peer-to-Peer communication, the server is no longer involved in answering each client's chat communication.

b. When in Chat mode, do the peers operate as clients, servers or both? Explain your answer.

During chat, the architecture transitions into Peer-to-Peer, and hence the peers operate as both servers and clients. They do not have a centralized server that mediates the communication between them, and thus each peer can act as both a client and server. Each peer initiates requests (client role) and handles incoming connections and responds (server role).

5. Is the client-server client communication using a persistent or non-persistent TCP connection? Explain your answer.

Non-persistent TCP connection
Each client-server request (REGISTER or BRIDGE) opens a connection, processes it, and then closes it. After the client exits a chat, it needs to re-initiate its register and bridge process to have a chat with another client once again.

6. Is your peer-peer client program using a persistent or non-persistent TCP connection? Explain your answer.

Persistent TCP connection
A continuous connection is maintained during the chat session until it is terminated by a /quit command or other exit signal (SIGINT).

7. Why do you think the specific type of TCP connection (persistent or non-persistent) was made for each of the two connections, client-server versus peer-peer?

Client-Server (Non-persistent)
- Non-persistent TCP connection allows for efficiency in handling multiple clients since connections don't remain open unnecessarily. After the clients /quit during a chat, they immediately terminate, and thus, the server has no reason to keep the connection alive.

Peer-to-Peer (Persistent)
- Persistent TCP connection ensures uninterrupted communication during the chat session without repeated connection overhead

8. Which aspect of your program follows half-duplex communication?

The chat alternates between writing and reading, similar to Walkie-Talkies, making it half-duplex. Only one peer can write while the other reads, ensuring no simultaneous data transmission.

9. Include screenshot(s) showing all three terminals, clearly displaying:
    a. Start up of your server

```
^Cmininet@mininet-vm:~$ python3 s.py --port=5555
Server is listening on 127.0.0.1:5555
REGISTER: Alice from 127.0.0.1:3000
BRIDGE: Alice 127.0.0.1:3000
REGISTER: Bob from 127.0.0.1:4000
BRIDGE: Bob 127.0.0.1:4000 Alice 127.0.0.1:3000
```

    b. Chat exchange of at least 3 lines between peers

```
mininet@mininet-vm:~$ python3 c.py --id="Bob" --port=4000 --server="127.0.0.1:5555"
Bob is running on 127.0.0.1:4000
/id
Bob
/register
/bridge
/chat
Hello Alice
Alice> How are you doing Bob?
Doing well alice. time for coffee?
Alice> sorry bob. I'm too busy
ok alice another time
/quit
Chat session ended
Exiting Program
```

```
mininet@mininet-vm:~$ python3 c.py --id="Alice" --port=3000 --server="127.0.0.1:5555"
Alice is running on 127.0.0.1:3000
/id
Alice
/register
/bridge
Incoming chat request from Bob 127.0.0.1:4000
Bob> Hello Alice
How are you doing Bob?
Bob> Doing well alice. time for coffee?
sorry bob. I'm too busy
Bob> ok alice another time
Bob has ended the chat session.
Exiting Program
```

10. Consider the simplifying assumptions for this assignment. Which simplification in particular needs to be addressed for the server to function as a real server?

    The assumption of only two clients limits scalability. For real-world usage, the server would need to handle multiple clients simultaneously and manage more complex data storage and retrieval.

```
mininet@mininet-vm:~/Desktop$ python3 s.py --port=5555
Server is listening on 127.0.0.1:5555
REGISTER: Alice from 127.0.0.1:3000
BRIDGE: Alice 127.0.0.1:3000
REGISTER: Bob from 127.0.0.1:4000
BRIDGE: Bob 127.0.0.1:4000 Alice 127.0.0.1:3000
```

```
mininet@mininet-vm:~/Desktop$ python3 p.py --id='Bob' --port=4000 --server='127.0.0.1:5555'
Bob is running on 127.0.0.1:4000
/id
Bob
/register
/bridge
/chat
Hello Alice
How are you doing, Bob_
Doing well Alice, time for coffee?
Sorry Bov, i'm too busy
Ok Alice another time
/quit
mininet@mininet-vm:~/Desktop$
```

```
mininet@mininet-vm:~/Desktop$ python3 p.py --id='Alice' --port=3000 --server='127.0.0.1:5555'
Alice is running on 127.0.0.1:3000
/id
Alice
/register
/bridge
Incoming chat request from Bob 127.0.0.1:4000
Hello Alice
How are you doing, Bob_
Doing well Alice, time for coffee?
Sorry Bov, i'm too busy
Ok Alice another time
mininet@mininet-vm:~/Desktop$
```